

Teoría de Lenguajes y Programación: Fundamentos de los Lenguajes Formales, Autómatas y Análisis Léxico-Sintáctico

Resumen

La Teoría de Lenguajes constituye una de las áreas fundamentales dentro de las Ciencias de la Computación, ya que proporciona las bases matemáticas y conceptuales necesarias para describir, analizar y reconocer los diferentes lenguajes utilizados en informática. Su estudio permite comprender cómo una computadora interpreta una secuencia de símbolos hasta convertirla en instrucciones ejecutables mediante procesos de análisis léxico y sintáctico. De acuerdo con el material del curso, esta disciplina estudia los mecanismos necesarios para la comunicación entre personas, computadoras y ambos sistemas, utilizando modelos formales que permiten representar de manera precisa la estructura de un lenguaje.

El presente trabajo desarrolla los principales conceptos relacionados con la Teoría de Lenguajes y Programación siguiendo el orden establecido en las diapositivas del curso. Se inicia con la definición de alfabetos, símbolos, palabras y lenguajes, para posteriormente introducir el concepto de gramática formal y su representación mediante la notación Backus-Naur (BNF). Posteriormente se estudia la clasificación de gramáticas propuesta por Noam Chomsky, la cual constituye una base para comprender los diferentes tipos de lenguajes formales.

Asimismo, se analizan las expresiones regulares como mecanismo para describir lenguajes regulares y los autómatas finitos deterministas y no deterministas como modelos matemáticos capaces de reconocer dichos lenguajes. Se explica el proceso de conversión de expresiones regulares hacia autómatas finitos no deterministas mediante el algoritmo de Thompson, así como la transformación de estos últimos en autómatas deterministas utilizando el método de construcción por subconjuntos.

Finalmente, el documento aborda el funcionamiento del análisis léxico mediante la herramienta Lex y el análisis sintáctico utilizando gramáticas LL(1), gramáticas LR(0) y el generador Bison. Estos elementos representan las etapas esenciales del proceso de compilación y permiten comprender cómo un compilador verifica la estructura de un programa fuente antes de traducirlo a lenguaje máquina.

Capítulo 1. Introducción

La evolución de la informática ha requerido el desarrollo de métodos formales capaces de representar la estructura de los lenguajes utilizados para la comunicación entre los seres humanos y las computadoras. Conforme los lenguajes de programación fueron incrementando su complejidad, surgió la necesidad de establecer reglas que permitieran describir de manera precisa la sintaxis de cada lenguaje y verificar automáticamente que los programas escritos por los usuarios cumplieran dichas reglas.

La Teoría de Lenguajes constituye la disciplina encargada de proporcionar estos fundamentos. Según el material del curso, esta área de las Ciencias de la Computación estudia el diseño y análisis de los lenguajes empleados en la comunicación entre personas, entre computadoras y entre ambos. Para ello utiliza modelos matemáticos capaces de representar formalmente los distintos tipos de lenguajes existentes.

Dentro de este contexto aparecen conceptos fundamentales como los alfabetos, los símbolos, las palabras y los lenguajes. Estos elementos representan la base sobre la cual posteriormente se construyen las gramáticas formales, las expresiones regulares y los autómatas finitos. Cada uno de estos modelos cumple una función específica dentro del análisis de un lenguaje y permite resolver diferentes problemas relacionados con el reconocimiento y procesamiento de cadenas de caracteres.

Las gramáticas formales permiten describir la estructura válida de un lenguaje mediante reglas de producción, mientras que las expresiones regulares proporcionan un mecanismo compacto para describir lenguajes regulares. Por otra parte, los autómatas finitos constituyen modelos matemáticos que permiten reconocer si una determinada cadena pertenece o no a un lenguaje específico. Estas herramientas forman parte de la base teórica sobre la que se construyen compiladores, intérpretes y analizadores sintácticos utilizados actualmente en el desarrollo de software.

El proceso de compilación requiere dividir el análisis de un programa en varias etapas. Inicialmente se realiza un análisis léxico encargado de identificar los diferentes componentes o tokens que conforman el código fuente. Posteriormente se lleva a cabo el análisis sintáctico, cuya finalidad consiste en comprobar que la secuencia de tokens cumple con las reglas definidas por la gramática del lenguaje. Para automatizar estos procesos existen herramientas especializadas como Lex y GNU Bison, ampliamente utilizadas para generar analizadores léxicos y sintácticos de forma automática.

El estudio de estos conceptos no solamente proporciona una comprensión más profunda acerca del funcionamiento interno de los compiladores, sino que también desarrolla la capacidad para modelar formalmente problemas relacionados con el procesamiento de información y el diseño de lenguajes de programación. Por ello, la Teoría de Lenguajes representa uno de los pilares fundamentales dentro de la formación de profesionales en Ciencias de la Computación e Ingeniería de Software.

Objetivo general

Analizar los fundamentos de la Teoría de Lenguajes y Programación mediante el estudio de los lenguajes formales, las gramáticas, las expresiones regulares, los autómatas finitos y los procesos de análisis léxico y sintáctico, con base exclusivamente en el material proporcionado durante el curso.

Objetivos específicos

- Describir los conceptos fundamentales de alfabeto, símbolo, palabra y lenguaje.
 - Explicar la estructura y funcionamiento de las gramáticas formales y la notación BNF.
 - Analizar la clasificación de gramáticas propuesta por Chomsky.
 - Comprender el funcionamiento de las expresiones regulares y su relación con los lenguajes regulares.
 - Explicar el funcionamiento de los autómatas finitos deterministas y no deterministas.
 - Describir el proceso de conversión entre expresiones regulares, AFND y AFD.
 - Analizar el funcionamiento del análisis léxico mediante Lex.
 - Explicar el análisis sintáctico utilizando gramáticas LL(1), LR(0) y GNU Bison.
-

Capítulo 2. Fundamentos de la Teoría de Lenguajes

2.1 Teoría de Lenguajes

La Teoría de Lenguajes constituye una disciplina perteneciente a las Ciencias de la Computación cuyo propósito principal consiste

en estudiar la estructura, representación y procesamiento de los distintos lenguajes utilizados para establecer comunicación entre personas, computadoras o ambos. Esta área proporciona los fundamentos necesarios para comprender cómo una computadora puede interpretar información escrita mediante reglas formales previamente establecidas.

El desarrollo de esta disciplina permitió establecer modelos matemáticos que describen el comportamiento de los lenguajes de programación y de otros sistemas de comunicación formal. Gracias a estos modelos es posible definir de manera precisa cuáles secuencias de símbolos pertenecen a un lenguaje y cuáles deben considerarse inválidas. Esta capacidad resulta indispensable durante la construcción de compiladores, intérpretes y analizadores de código fuente.

Dentro de la Teoría de Lenguajes, cada concepto depende del anterior. Primero se definen los símbolos que pueden utilizarse; posteriormente dichos símbolos forman alfabetos; los alfabetos permiten construir palabras; finalmente, el conjunto de palabras válidas constituye un lenguaje. Esta organización jerárquica facilita el estudio formal de cualquier sistema de comunicación.

2.2 Alfabetos

El concepto más elemental dentro de la Teoría de Lenguajes es el **alfabeto**. Un alfabeto se define como un conjunto finito y no vacío cuyos elementos reciben el nombre de símbolos. Estos símbolos representan las unidades básicas con las cuales pueden construirse palabras y, posteriormente, lenguajes completos.

Generalmente los alfabetos se representan mediante la letra griega Σ (sigma). Dependiendo del problema que se desea modelar, el contenido del alfabeto puede variar considerablemente. Por ejemplo, un alfabeto puede estar formado por las letras del idioma inglés, por los dígitos del sistema decimal o incluso por operadores utilizados en un lenguaje de programación. Las diapositivas presentan ejemplos como:

- $\Sigma_1 = \{A, B, C, \dots, Z\}$
- $\Sigma_2 = \{0, 1\}$
- $\Sigma_3 = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, .\}$
- $\Sigma_4 = \{/, \backslash\}$

La elección del alfabeto determina completamente las palabras que podrán construirse posteriormente. Si un símbolo no pertenece al alfabeto definido, entonces no puede formar parte del lenguaje que se está describiendo.

Desde el punto de vista computacional, el alfabeto constituye el conjunto de entradas que un autómata o un compilador puede reconocer durante el procesamiento de una cadena.

2.3 Símbolos

Los **símbolos** representan los elementos individuales que conforman un alfabeto. Cada símbolo constituye la unidad mínima de información utilizada para construir palabras dentro de un lenguaje formal. A diferencia de una palabra, un símbolo no puede dividirse en unidades más pequeñas con significado dentro del sistema considerado.

Dependiendo del contexto, un símbolo puede representar una letra, un número, un operador aritmético, un signo de puntuación o cualquier otro carácter definido previamente en el alfabeto. Por ejemplo, si se considera el alfabeto $\Sigma = \{a, b\}$, únicamente los caracteres **a** y **b** podrán utilizarse para construir palabras válidas.

En programación ocurre exactamente el mismo principio. Los lenguajes establecen un conjunto específico de caracteres válidos que posteriormente serán agrupados para formar identificadores, palabras reservadas, constantes y operadores.

2.4 Palabras

Una **palabra** es una secuencia finita formada exclusivamente mediante símbolos pertenecientes a un mismo alfabeto. Dicho de otra manera, una palabra consiste en una concatenación ordenada de símbolos válidos.

Las diapositivas presentan ejemplos sencillos que ilustran este concepto:

- **JUAN** constituye una palabra construida utilizando el alfabeto de letras mayúsculas.
- **1234** constituye una palabra formada mediante símbolos pertenecientes al alfabeto decimal.

Es importante observar que una palabra no necesita poseer significado lingüístico. Desde el punto de vista formal, basta con que esté compuesta únicamente por símbolos pertenecientes al alfabeto definido. Por esta razón, secuencias aparentemente sin sentido pueden representar palabras completamente válidas dentro de un lenguaje formal.

En los lenguajes de programación, ejemplos de palabras incluyen nombres de variables, constantes numéricas, operadores o palabras reservadas reconocidas posteriormente durante el análisis léxico.

2.5 Longitud de una palabra

Una característica importante de cualquier palabra es su **longitud**, definida como el número total de símbolos que la conforman. La longitud suele representarse mediante la notación $|x|$, donde x corresponde a la palabra analizada.

Por ejemplo:

- Si $x = \text{computadora}$, entonces $|x|$ corresponde al número de caracteres que integran dicha palabra.
- Si $y = 12345$, entonces $|y| = 5$.

La longitud permite comparar palabras y constituye un concepto ampliamente utilizado en el análisis matemático de lenguajes

formales, ya que numerosos algoritmos dependen directamente del número de símbolos presentes en la entrada.

2.6 Palabra vacía

Dentro de todos los lenguajes existe un caso particular conocido como **palabra vacía**. Se trata de una palabra cuya longitud es igual a cero y que tradicionalmente se representa mediante la letra griega λ (lambda). En algunos contextos también puede representarse mediante ε (épsilon), dependiendo de la notación utilizada.

Aunque no contiene símbolos, la palabra vacía sigue considerándose una palabra válida dentro del lenguaje. Su existencia resulta especialmente importante en la definición de expresiones regulares, gramáticas formales y autómatas finitos, ya que permite representar situaciones en las cuales una transición o una producción puede realizarse sin consumir ningún símbolo de entrada.

Este concepto aparecerá nuevamente en capítulos posteriores al estudiar las transiciones ε de los autómatas no deterministas y las producciones vacías utilizadas en determinadas gramáticas.

2.7 Lenguajes

Una vez definidos los conceptos anteriores es posible comprender la noción de **lenguaje**. Un lenguaje puede entenderse como un conjunto de palabras construidas utilizando los símbolos de un alfabeto determinado y que cumplen reglas específicas de formación. Dicho conjunto puede contener un número finito o infinito de palabras, dependiendo de las reglas que lo describan.

Desde una perspectiva computacional, un lenguaje representa todas las cadenas que un sistema es capaz de reconocer como válidas. Por ejemplo, un lenguaje de programación define todas las instrucciones que un compilador puede aceptar, mientras que un

autómata finito reconoce únicamente aquellas cadenas que pertenecen al lenguaje para el cual fue diseñado.

La descripción formal de un lenguaje requiere mecanismos matemáticos que permitan expresar sus reglas de construcción. Precisamente por esta razón surgen las gramáticas formales, las expresiones regulares y los autómatas, herramientas que serán desarrolladas en los capítulos siguientes.

Capítulo 3. Gramáticas Formales

3.1 Introducción

Una vez comprendidos los conceptos de alfabeto, símbolo, palabra y lenguaje, el siguiente paso consiste en estudiar el mecanismo que permite definir de manera formal la estructura de un lenguaje. En las Ciencias de la Computación, esta función es desempeñada por las **gramáticas formales**, las cuales establecen un conjunto de reglas capaces de describir todas las cadenas válidas de un lenguaje.

Las gramáticas constituyen uno de los pilares de la Teoría de Lenguajes porque permiten representar matemáticamente la sintaxis de un lenguaje de programación. Gracias a ellas es posible construir analizadores sintácticos que verifican si un programa cumple con las reglas establecidas antes de proceder a su compilación.

Mientras que las expresiones regulares son adecuadas para describir estructuras relativamente simples, las gramáticas permiten representar construcciones mucho más complejas, incluyendo expresiones anidadas, estructuras condicionales y bloques de instrucciones.

3.2 Concepto de gramática formal

Una **gramática** describe la estructura que deben seguir las palabras y las frases de un lenguaje. En el ámbito de la computación, una gramática formal especifica cómo pueden combinarse los diferentes símbolos para generar cadenas válidas. De esta forma, todas las palabras aceptadas por un lenguaje pueden obtenerse mediante la aplicación sucesiva de reglas de producción.

Formalmente, una gramática se representa mediante la cuádrupla:

$$G = (\Sigma T, \Sigma N, S, P)$$

donde:

- ΣT representa el conjunto de símbolos terminales.
- ΣN representa el conjunto de símbolos no terminales.
- S corresponde al símbolo inicial o axioma.
- P representa el conjunto de reglas de producción.

Además, las diapositivas establecen que los conjuntos de símbolos terminales y no terminales son disjuntos, es decir, no comparten elementos entre sí.

Esta representación permite describir formalmente cualquier lenguaje mediante un conjunto finito de reglas.

3.3 Componentes de una gramática

Símbolos terminales

Los símbolos terminales constituyen los elementos que finalmente aparecen en las cadenas generadas por la gramática. Una vez alcanzados estos símbolos, el proceso de derivación concluye, ya que no existen reglas adicionales que permitan sustituirlos.

En un lenguaje de programación, los símbolos terminales pueden corresponder a identificadores, operadores, palabras reservadas o constantes reconocidas por el analizador léxico.

Símbolos no terminales

Los símbolos no terminales representan categorías intermedias utilizadas durante el proceso de construcción de una cadena. A diferencia de los terminales, estos símbolos pueden sustituirse mediante reglas de producción hasta obtener únicamente símbolos terminales.

Por ejemplo, un símbolo no terminal puede representar una expresión, una instrucción o un número dentro de una gramática determinada.

Símbolo inicial

Toda gramática posee un símbolo especial denominado **símbolo inicial** o **axioma**, representado por **S**. Este símbolo constituye el punto de partida del proceso de derivación y a partir de él se generan todas las cadenas pertenecientes al lenguaje.

El símbolo inicial aparece únicamente al comenzar la derivación y representa el nivel más alto de la estructura sintáctica del lenguaje.

Reglas de producción

Las reglas de producción indican cómo un símbolo no terminal puede sustituirse por una combinación de símbolos terminales y no terminales. Estas reglas determinan completamente la estructura del lenguaje y establecen todas las derivaciones válidas que pueden realizarse.

Cada producción representa una transformación que acerca progresivamente la derivación hacia una cadena formada únicamente por símbolos terminales.

3.4 Ejemplo de una gramática

Las diapositivas presentan una gramática sencilla destinada a generar números enteros formados por uno o más dígitos. En este ejemplo, los símbolos terminales corresponden a los dígitos del sistema decimal, mientras que los símbolos no terminales son **N** y **C**. El símbolo inicial es **N**, y las reglas de producción permiten

construir números de cualquier longitud mediante la concatenación de dígitos.

Este ejemplo ilustra cómo una gramática puede describir un conjunto potencialmente infinito de cadenas utilizando únicamente un número reducido de reglas de producción. Aunque el conjunto de números enteros es muy amplio, la gramática logra representarlo de forma compacta mediante derivaciones sucesivas.

Este principio constituye una de las principales ventajas de las gramáticas formales: permiten describir lenguajes complejos utilizando una cantidad limitada de reglas.

3.5 Notación Backus-Naur (BNF)

A medida que las gramáticas aumentan su complejidad, resulta conveniente utilizar una notación que facilite la escritura de las reglas de producción. Con este propósito surge la **Backus-Naur Form (BNF)**, una notación abreviada empleada para representar gramáticas formales de manera más clara y organizada.

La principal ventaja de BNF consiste en que varias producciones con el mismo lado izquierdo pueden agruparse utilizando el operador de alternativa. Esto reduce la cantidad de reglas escritas sin modificar el significado de la gramática.

Gracias a esta representación, las especificaciones de numerosos lenguajes de programación pueden describirse de manera compacta y fácil de interpretar.

Actualmente, BNF continúa utilizándose para documentar la sintaxis de distintos lenguajes y constituye un estándar ampliamente reconocido dentro del desarrollo de compiladores.

3.6 Clasificación de las gramáticas

Las gramáticas no poseen todas la misma capacidad para describir lenguajes. Algunas permiten representar únicamente estructuras muy simples, mientras que otras pueden expresar construcciones considerablemente más complejas. Con el propósito de organizar

estas diferencias, Noam Chomsky propuso una clasificación conocida como la **Jerarquía de Chomsky**, desarrollada en 1956. Esta clasificación organiza los lenguajes y las gramáticas de acuerdo con su poder de representación.

La jerarquía establece cuatro niveles principales:

- **Tipo 0:** Gramáticas no restringidas.
- **Tipo 1:** Gramáticas sensibles al contexto.
- **Tipo 2:** Gramáticas libres de contexto.
- **Tipo 3:** Gramáticas regulares.

Cada nivel incluye al siguiente, de manera que las gramáticas más generales poseen una mayor capacidad descriptiva, aunque también requieren mecanismos de análisis más complejos.

3.7 Gramáticas libres de contexto

Dentro de la clasificación anterior, las **gramáticas libres de contexto** poseen una importancia especial para el desarrollo de compiladores. En ellas, la sustitución de un símbolo no terminal depende únicamente del propio símbolo y no del contexto en el cual aparece.

Esta característica permite construir analizadores sintácticos eficientes capaces de reconocer la estructura de la mayoría de los lenguajes de programación modernos. Los métodos LL(1) y LR(0), estudiados en capítulos posteriores, pertenecen precisamente a esta categoría de gramáticas.

Gracias a su equilibrio entre expresividad y facilidad de análisis, las gramáticas libres de contexto se han convertido en el modelo predominante para describir la sintaxis de los lenguajes de programación.

Capítulo 4. Expresiones Regulares

4.1 Introducción

Después de estudiar las gramáticas formales, resulta necesario conocer otro mecanismo ampliamente utilizado para describir lenguajes: las **expresiones regulares**. Estas constituyen una forma compacta de representar conjuntos de cadenas pertenecientes a un lenguaje regular y permiten especificar patrones que posteriormente serán reconocidos por analizadores léxicos y autómatas finitos.

Las expresiones regulares representan uno de los primeros niveles de abstracción utilizados en el diseño de compiladores, ya que facilitan la definición de los diferentes componentes léxicos que conforman un lenguaje de programación. Gracias a ellas es posible describir números, identificadores, operadores, palabras reservadas y otros elementos sin necesidad de enumerar todas las cadenas posibles.

Además de su aplicación en compiladores, las expresiones regulares permiten modelar lenguajes regulares mediante una notación sencilla, basada en un conjunto reducido de operadores que pueden combinarse para construir descripciones cada vez más complejas.

4.2 Definición de expresión regular

Las diapositivas definen una **expresión regular** como una manera simple y concisa de describir lenguajes regulares utilizando únicamente los símbolos pertenecientes al alfabeto sobre el cual están contruidos. En lugar de enumerar todas las palabras que forman un lenguaje, una expresión regular permite representarlas mediante una estructura simbólica mucho más compacta.

El significado de una expresión regular no depende únicamente de los símbolos que contiene, sino también de los operadores utilizados para relacionarlos. Estos operadores permiten construir expresiones capaces de representar un número potencialmente infinito de cadenas.

Desde el punto de vista formal, toda expresión regular describe un lenguaje, es decir, un conjunto de palabras que cumplen determinadas reglas de construcción.

4.3 Definición recursiva

Las expresiones regulares pueden definirse de manera recursiva. El material del curso distingue dos grupos de reglas: los **casos base** y el **paso recursivo**.

Casos base

Inicialmente existen tres expresiones fundamentales:

- ϵ , que representa el lenguaje formado únicamente por la palabra vacía.
- $\emptyset$, que representa el lenguaje vacío, es decir, aquel que no contiene ninguna palabra.
- a , donde a representa un símbolo perteneciente al alfabeto y genera el lenguaje formado exclusivamente por dicho símbolo.

Estas expresiones constituyen los bloques básicos a partir de los cuales pueden construirse expresiones considerablemente más complejas.

Paso recursivo

Una vez definidos los casos base, pueden aplicarse diferentes operaciones para construir nuevas expresiones regulares.

Las diapositivas indican que, si r y s representan expresiones regulares, entonces también son expresiones válidas:

- $(r \mid s)$
- (rs)
- (r)
- el uso de paréntesis para agrupar expresiones.

Gracias a estas operaciones es posible describir prácticamente cualquier lenguaje regular utilizando únicamente unos cuantos operadores.

4.4 Operaciones fundamentales

Unión

La **unión** permite seleccionar entre dos alternativas diferentes. Se representa mediante el operador vertical ($|$) y genera un lenguaje formado por todas las palabras pertenecientes a cualquiera de las expresiones involucradas.

Por ejemplo:

$$L(a|b) = \{a, b\}$$

En este caso el lenguaje acepta cualquiera de los dos símbolos especificados.

Este operador resulta especialmente útil cuando un lenguaje admite varias opciones para ocupar una misma posición dentro de una cadena.

Concatenación

La **concatenación** consiste en colocar una expresión inmediatamente después de otra, formando palabras de mayor longitud.

Por ejemplo:

$$L(ab)$$

representa el lenguaje formado por la palabra **ab**, obtenida al colocar primero el símbolo **a** y posteriormente el símbolo **b**.

Las diapositivas muestran además ejemplos en los cuales la concatenación se aplica entre conjuntos completos de palabras, generando todas las combinaciones posibles entre ambos lenguajes.

Esta operación constituye uno de los mecanismos más utilizados para describir la estructura secuencial de los lenguajes de programación.

Cerradura de Kleene

La tercera operación fundamental corresponde a la **cerradura de Kleene**, representada mediante el símbolo (*).

Su finalidad consiste en indicar que una expresión puede repetirse cero o más veces.

Por ejemplo:

a^*

representa el conjunto:

$\{\epsilon, a, aa, aaa, aaaa, \dots\}$

Es decir, la letra **a** puede aparecer cualquier cantidad de veces, incluyendo el caso en que no aparezca ninguna.

La cerradura de Kleene constituye una herramienta extremadamente poderosa porque permite representar lenguajes infinitos mediante expresiones relativamente pequeñas.

4.5 Interpretación de lenguajes

Cada expresión regular posee un lenguaje asociado.

Las diapositivas muestran que:

$$L(r) = L(r)^*$$

es decir, el lenguaje descrito por una expresión con cerradura corresponde a todas las concatenaciones posibles del lenguaje original.

Asimismo, se presentan ejemplos donde intervienen simultáneamente la unión y la cerradura, demostrando cómo una expresión aparentemente sencilla puede describir un número muy elevado de cadenas diferentes.

Este principio resulta fundamental para comprender posteriormente cómo los analizadores léxicos reconocen patrones dentro de un programa fuente.

4.6 Ejemplos de expresiones regulares

Las diapositivas incluyen distintos ejemplos cuyo objetivo consiste en ilustrar el funcionamiento de los operadores descritos anteriormente.

Uno de ellos muestra cómo la expresión:

$(a \mid bb)^*$

genera un lenguaje que contiene la palabra vacía, además de todas las combinaciones posibles obtenidas mediante la repetición de **a** o **bb**.

Otro ejemplo desarrolla expresiones como:

$a^* \mid b^*$

demostrando que el lenguaje obtenido corresponde a la unión entre todas las cadenas formadas exclusivamente por la letra **a** y todas aquellas formadas únicamente por la letra **b**.

Estos ejemplos permiten comprender cómo la combinación de unos cuantos operadores genera lenguajes considerablemente más complejos que los casos básicos.

4.7 Aplicación práctica

Después de comprender la sintaxis de las expresiones regulares, las diapositivas presentan un ejercicio completo donde se solicita construir una expresión capaz de reconocer todas las cadenas binarias que contienen tres ceros consecutivos.

La solución propuesta utiliza la expresión:

$(0 \mid 1)000(0 \mid 1)$

Esta representación garantiza que la subcadena **000** aparezca en cualquier posición dentro de la palabra, permitiendo cualquier combinación de ceros y unos antes o después de ella.

Este ejercicio sirve como punto de partida para los capítulos siguientes, donde dicha expresión regular será transformada primero en un **Autómata Finito No Determinista** mediante el

algoritmo de Thompson y posteriormente en un **Autómata Finito Determinista** mediante el método de subconjuntos.

De esta manera puede observarse que las expresiones regulares no constituyen únicamente una notación matemática, sino también el punto inicial para la construcción de mecanismos automáticos capaces de reconocer lenguajes.

Capítulo 5. Autómatas Finitos

5.1 Introducción

En los capítulos anteriores se estudiaron los lenguajes formales y las expresiones regulares como mecanismos para describir conjuntos de cadenas. Sin embargo, además de describir un lenguaje, es necesario contar con un procedimiento que permita determinar automáticamente si una cadena pertenece o no a dicho lenguaje. Esta función es realizada por los **autómatas finitos**, modelos matemáticos capaces de reconocer lenguajes regulares mediante una secuencia de estados y transiciones.

Los autómatas constituyen una herramienta fundamental en la Teoría de Lenguajes porque representan el mecanismo de reconocimiento utilizado posteriormente durante el análisis léxico de un compilador. Cada vez que un programa procesa un identificador, una constante numérica o una palabra reservada, internamente se emplean modelos equivalentes a autómatas finitos para verificar que dichas cadenas cumplen con la estructura definida por el lenguaje.

El estudio de los autómatas también demuestra la estrecha relación existente entre expresiones regulares y lenguajes formales, ya que toda expresión regular puede convertirse en un autómata equivalente capaz de reconocer exactamente el mismo lenguaje.

5.2 Concepto de autómata finito

Las diapositivas introducen tres tipos de aceptadores o autómatas:

- Autómatas Finitos Deterministas (AFD).
- Autómatas Finitos No Deterministas (AFND).
- Autómatas No Deterministas con transiciones ϵ .

Asimismo, se establece que estos tres modelos poseen la misma capacidad de reconocimiento, es decir, son equivalentes desde el punto de vista de los lenguajes que pueden aceptar.

Un autómata puede entenderse como una máquina de estados finitos. Durante su funcionamiento recibe una secuencia de símbolos pertenecientes a un alfabeto y cambia de estado conforme a una función de transición previamente definida. Una vez procesada toda la cadena, el autómata determina si ésta pertenece o no al lenguaje representado.

Cada estado refleja una situación específica del proceso de reconocimiento, mientras que las transiciones indican cómo evoluciona el sistema cuando recibe un símbolo de entrada.

5.3 Autómatas Finitos Deterministas (AFD)

El primer modelo estudiado corresponde al **Autómata Finito Determinista**.

Las diapositivas indican que un AFD se caracteriza porque para cada estado y cada símbolo del alfabeto existe, como máximo, una transición posible. Esto significa que el comportamiento del sistema siempre está completamente determinado y no existe ambigüedad durante el reconocimiento de una cadena.

Formalmente, un AFD se representa mediante la quintupla:

$$M = (Q, \Sigma, q_0, F, \delta)$$

donde:

- Q representa el conjunto finito de estados.
- Σ corresponde al alfabeto de entrada.

- q_0 representa el estado inicial.
- F contiene los estados de aceptación.
- δ es la función de transición.

Cada uno de estos elementos participa en el proceso de reconocimiento y determina completamente el comportamiento del autómata.

Función de transición

La función de transición constituye el mecanismo mediante el cual el autómata cambia de estado cuando recibe un símbolo determinado.

Las diapositivas señalan que esta función suele representarse mediante una tabla donde cada fila corresponde a un estado y cada columna representa un símbolo del alfabeto. En cada intersección aparece el siguiente estado al cual debe desplazarse el autómata.

Esta representación facilita el diseño y análisis de autómatas de mayor tamaño.

Ejemplo de AFD

Como ejemplo, el material presenta una máquina expendedora de refrescos que acepta monedas de diferentes denominaciones.

En este caso, cada estado representa la cantidad acumulada de dinero, mientras que las transiciones corresponden a la inserción de nuevas monedas. Cuando se alcanza el estado final, la máquina entrega el producto solicitado.

Este ejemplo demuestra que los autómatas no sólo pueden emplearse para reconocer lenguajes, sino también para modelar el comportamiento de diversos sistemas discretos.

5.4 Autómatas Finitos No Deterministas (AFND)

El segundo modelo estudiado corresponde al **Autómata Finito No Determinista**.

A diferencia del AFD, un AFND permite que desde un mismo estado existan varias transiciones asociadas al mismo símbolo de entrada o incluso que algunas transiciones no existan. Asimismo, puede incluir transiciones etiquetadas con cadenas completas o con la palabra vacía.

Como consecuencia, durante el procesamiento de una cadena pueden existir múltiples caminos posibles.

Desde el punto de vista matemático, un AFND también se representa mediante una quintupla semejante al AFD:

$(Q, \Sigma, q_0, F, \Delta)$

La diferencia principal radica en que la relación de transición Δ puede producir varios estados de destino para una misma entrada.

Ventajas del AFND

Las diapositivas destacan que el diseño de un AFND suele ser considerablemente más sencillo que el de un AFD.

Por ejemplo, para reconocer una determinada subcadena dentro de una palabra basta con añadir nuevas transiciones, sin necesidad de modificar completamente la estructura del autómata. Esta característica simplifica notablemente la construcción inicial del modelo.

No obstante, esta flexibilidad introduce una mayor complejidad durante el reconocimiento efectivo de las cadenas, ya que pueden existir múltiples recorridos posibles.

5.5 Conversión de una Expresión Regular a un AFND

Una de las contribuciones más importantes estudiadas en las diapositivas consiste en demostrar que toda expresión regular puede transformarse en un autómata equivalente.

Para realizar esta transformación se utiliza el **Algoritmo de Thompson**, cuyo objetivo consiste en construir un AFND con transiciones ϵ capaz de reconocer exactamente el mismo lenguaje descrito por la expresión regular.

Este algoritmo representa uno de los procedimientos fundamentales en el diseño de compiladores.

Algoritmo de Thompson

El algoritmo descompone la expresión regular en partes más pequeñas y construye un autómata elemental para cada una de ellas.

Posteriormente dichos autómatas se combinan utilizando transiciones ϵ hasta obtener un único autómata equivalente a toda la expresión regular.

Las diapositivas indican que el procedimiento se basa en tres operaciones principales:

- símbolo individual;
- unión;
- concatenación;
- cerradura de Kleene.

Cada una posee una construcción específica.

Ejemplo de Thompson

Como ejemplo se analiza la expresión:

$(a|b)a^*$

El procedimiento comienza construyendo el autómata correspondiente a la unión **$(a|b)$** .

Posteriormente se genera el autómata asociado a la expresión **a^*** .

Finalmente ambos fragmentos se conectan mediante concatenación para obtener un único AFND que reconoce completamente el lenguaje representado por la expresión regular.

Este ejemplo permite observar cómo una expresión regular termina convirtiéndose en una máquina capaz de reconocer cadenas automáticamente.

5.6 Conversión de AFND a AFD

Aunque los AFND resultan sencillos de construir, en la práctica suele ser más conveniente utilizar autómatas deterministas durante el reconocimiento de cadenas.

Por esta razón las diapositivas presentan el **método de construcción por subconjuntos**, cuyo objetivo consiste en transformar un AFND en un AFD equivalente.

Durante este procedimiento cada estado del nuevo AFD representa un conjunto de estados pertenecientes al AFND original.

Este enfoque permite eliminar el no determinismo sin modificar el lenguaje aceptado por el autómata.

Método de subconjuntos

El procedimiento comienza identificando los conjuntos de estados alcanzables mediante las transiciones ϵ .

Posteriormente se generan nuevos estados agrupando todas las posibles configuraciones del AFND.

Finalmente se construye una nueva función de transición completamente determinista.

Las diapositivas muestran paso a paso la construcción de estos llamados "mega-estados" hasta obtener el AFD equivalente.

5.7 Relación entre expresiones regulares y autómatas

El estudio realizado a lo largo de este capítulo permite establecer una relación directa entre tres conceptos fundamentales:

Expresión Regular $\rightarrow$ AFND $\rightarrow$ AFD

Cada uno representa exactamente el mismo lenguaje, aunque utilizando mecanismos diferentes.

La expresión regular describe el lenguaje mediante operadores algebraicos.

El AFND facilita la construcción inicial del reconocedor.

Finalmente, el AFD proporciona una implementación eficiente para reconocer cadenas durante el análisis léxico.

Esta equivalencia constituye uno de los principios más importantes de la Teoría de Lenguajes y explica por qué los analizadores léxicos pueden generarse automáticamente a partir de expresiones regulares.

Capítulo 6. Análisis Léxico y la Herramienta Lex

6.1 Introducción

En los capítulos anteriores se estudiaron las expresiones regulares y los autómatas finitos como herramientas para describir y reconocer lenguajes regulares. Sin embargo, estos conceptos adquieren una aplicación práctica durante la primera etapa del proceso de compilación: el **análisis léxico**. Esta fase tiene como objetivo leer el programa fuente y dividirlo en unidades básicas llamadas *tokens*, que posteriormente serán utilizadas por el analizador sintáctico.

El análisis léxico constituye la primera interacción entre un compilador y el código escrito por el programador. Su correcto funcionamiento resulta indispensable, ya que cualquier error detectado en esta etapa impide que el proceso de compilación continúe.

Para automatizar la construcción de analizadores léxicos existen herramientas especializadas como **Lex**, que permiten generar

automáticamente el código necesario a partir de un conjunto de reglas descritas mediante expresiones regulares.

6.2 El proceso de análisis léxico

Las diapositivas explican que el análisis léxico es la primera fase de un compilador y tiene como propósito identificar los elementos que conforman un programa fuente. Durante este proceso, el texto del programa es recorrido carácter por carácter hasta formar unidades con significado dentro del lenguaje.

Entre los componentes que normalmente reconoce un analizador léxico se encuentran:

- identificadores;
- palabras reservadas;
- constantes numéricas;
- operadores;
- signos de puntuación;
- delimitadores.

Cada uno de estos elementos recibe el nombre de **token**.

En esta etapa todavía no se analiza la estructura del programa; únicamente se determina qué tipo de componente representa cada secuencia de caracteres.

6.3 Tokens

Los **tokens** constituyen la salida principal del análisis léxico. Un token representa una categoría léxica reconocida dentro del programa fuente y será utilizada posteriormente durante el análisis sintáctico.

Por ejemplo, una instrucción sencilla puede dividirse en varios tokens consecutivos:

```
entero contador = 10;
```

Durante el análisis léxico, cada fragmento se identifica según su función dentro del lenguaje. Así, una palabra reservada, un identificador, un operador de asignación, una constante y un delimitador son reconocidos como unidades independientes.

Este proceso permite transformar una secuencia continua de caracteres en una secuencia estructurada de tokens, facilitando el trabajo de las etapas posteriores del compilador.

6.4 Relación entre expresiones regulares y análisis léxico

Las expresiones regulares estudiadas anteriormente constituyen el mecanismo utilizado para definir la estructura de cada token.

Por ejemplo, un identificador puede describirse mediante una expresión regular que indique que debe comenzar con una letra y continuar con letras o dígitos, mientras que una constante numérica puede representarse mediante otra expresión distinta.

Las diapositivas muestran precisamente esta relación entre:

- expresiones regulares;
- autómatas finitos;
- analizadores léxicos.

En consecuencia, el reconocimiento de tokens no se realiza de manera arbitraria, sino mediante autómatas construidos automáticamente a partir de expresiones regulares.

Esta relación demuestra la aplicación práctica de los conceptos desarrollados en los capítulos anteriores.

6.5 La herramienta Lex

Las diapositivas presentan **Lex** como una herramienta destinada a generar analizadores léxicos de forma automática. En lugar de programar manualmente un autómatas finito, el desarrollador únicamente debe especificar un conjunto de reglas escritas mediante expresiones regulares y las acciones asociadas a cada una de ellas.

A partir de esta información, Lex genera un programa en lenguaje C que implementa el analizador léxico correspondiente.

Este enfoque reduce considerablemente el tiempo de desarrollo y disminuye la probabilidad de errores durante la implementación.

6.6 Estructura de un archivo Lex

El material del curso explica que un archivo de Lex se encuentra dividido en **tres secciones principales**, separadas mediante el símbolo **%%**.

Estas secciones son:

Declaraciones

En esta parte se incluyen definiciones, bibliotecas y declaraciones que serán utilizadas durante el análisis léxico.

También pueden declararse expresiones regulares reutilizables para simplificar la escritura del programa.

Reglas

La segunda sección constituye el núcleo del analizador léxico.

Aquí se escriben las expresiones regulares que describen cada token junto con la acción que deberá ejecutarse cuando dicho patrón sea reconocido.

Lex evalúa estas reglas conforme procesa el archivo fuente.

Cuando una expresión coincide con la entrada actual, ejecuta inmediatamente la acción correspondiente.

Código auxiliar

Finalmente, la tercera sección contiene funciones adicionales escritas en lenguaje C.

Estas funciones complementan el funcionamiento del analizador y pueden utilizarse para realizar tareas específicas durante el procesamiento del programa.

6.7 Funcionamiento interno de Lex

Aunque el usuario únicamente escribe expresiones regulares, internamente Lex realiza un proceso considerablemente más complejo.

Las diapositivas muestran que el generador transforma automáticamente cada expresión regular en un autómata finito equivalente, el cual será utilizado posteriormente para reconocer los diferentes tokens presentes en el programa fuente.

Este procedimiento demuestra nuevamente la estrecha relación existente entre:

- expresiones regulares;
- autómatas;
- compiladores.

Desde el punto de vista del desarrollador, todo este proceso ocurre de manera transparente.

6.8 Ventajas de utilizar Lex

La utilización de Lex ofrece diversas ventajas durante el desarrollo de compiladores.

Entre las más importantes pueden mencionarse:

- automatiza la construcción del analizador léxico;
- disminuye la cantidad de código que debe escribirse manualmente;

- facilita el mantenimiento del proyecto;
- permite modificar fácilmente las reglas del lenguaje;
- aprovecha expresiones regulares para describir patrones complejos.

Estas características explican por qué Lex ha sido ampliamente utilizado durante décadas en cursos de compiladores y en proyectos relacionados con el procesamiento de lenguajes.

6.9 Relación con las etapas posteriores del compilador

El análisis léxico representa únicamente la primera fase del proceso de compilación.

Una vez obtenidos los tokens, éstos son enviados al **analizador sintáctico**, cuya función consiste en comprobar si la secuencia obtenida cumple con las reglas establecidas por la gramática del lenguaje.

En consecuencia, el trabajo del analizador sintáctico depende completamente de la calidad del análisis léxico.

Si un token es identificado incorrectamente, el resto del proceso también producirá resultados erróneos.

Por esta razón ambas etapas funcionan de manera estrechamente coordinada dentro de cualquier compilador moderno.

Capítulo 7. Análisis Sintáctico y Generación Automática de Parsers

7.1 Introducción

Una vez concluido el análisis léxico, el compilador dispone de una secuencia de tokens que representan los diferentes componentes del programa fuente. Sin embargo, identificar

correctamente estos elementos no garantiza que el programa esté escrito conforme a las reglas del lenguaje. Es necesario comprobar además que dichos tokens aparezcan en un orden válido y formen estructuras sintácticas correctas. Esta tarea corresponde al **análisis sintáctico**, segunda etapa fundamental del proceso de compilación.

El análisis sintáctico utiliza las gramáticas formales estudiadas anteriormente para verificar la estructura del programa. Mediante este proceso, el compilador determina si la secuencia de tokens puede generarse a partir de la gramática del lenguaje y, en caso contrario, detecta errores relacionados con la sintaxis.

Además de validar la estructura del programa, el analizador sintáctico construye representaciones internas que posteriormente serán utilizadas durante las fases de análisis semántico y generación de código.

7.2 Árboles de derivación

Las diapositivas presentan los **árboles de derivación** como una representación gráfica del proceso mediante el cual una gramática genera una cadena. Cada nodo del árbol corresponde a un símbolo de la gramática, mientras que las ramas representan la aplicación de reglas de producción.

El árbol comienza siempre con el símbolo inicial de la gramática y, mediante aplicaciones sucesivas de reglas de producción, llega finalmente a una secuencia formada únicamente por símbolos terminales. Estos símbolos aparecen en las hojas del árbol y, leídos de izquierda a derecha, forman la cadena reconocida.

Los árboles de derivación permiten visualizar claramente la estructura jerárquica de una expresión y constituyen una herramienta fundamental para comprender cómo interpreta un compilador las instrucciones escritas por el programador.

Importancia de los árboles de derivación

Los árboles de derivación no sólo representan el proceso de reconocimiento de una cadena, sino que también permiten identificar la relación existente entre los diferentes componentes de una expresión.

Por ejemplo, una operación aritmética puede involucrar varios operadores y operandos. El árbol muestra de manera explícita qué operaciones deben realizarse primero y cuáles dependen de los resultados obtenidos previamente.

Esta representación evita ambigüedades y facilita la evaluación correcta de expresiones complejas.

7.3 Análisis sintáctico descendente

Uno de los enfoques estudiados para construir analizadores sintácticos es el **análisis descendente**, en el cual el proceso comienza desde el símbolo inicial de la gramática e intenta derivar la cadena de entrada mediante la aplicación sucesiva de reglas de producción.

Durante este procedimiento, el analizador predice cuál producción debe aplicarse observando el siguiente token de entrada. Si la producción seleccionada coincide con la secuencia recibida, el análisis continúa; de lo contrario, se detecta un error sintáctico.

Este enfoque resulta relativamente sencillo de implementar y constituye la base de las gramáticas LL(1).

7.4 Gramáticas LL(1)

Las diapositivas introducen las **gramáticas LL(1)** como un tipo particular de gramáticas libres de contexto que pueden analizarse utilizando un único símbolo de anticipación (*lookahead*).

La notación LL(1) indica las principales características del método:

- el análisis se realiza de **izquierda a derecha** sobre la cadena de entrada;
- la derivación utilizada corresponde a la **más a la izquierda**;
- únicamente se observa **un símbolo de anticipación** para decidir la producción que debe aplicarse.

Estas propiedades permiten construir analizadores eficientes siempre que la gramática cumpla determinadas restricciones.

Tabla de análisis LL(1)

Las diapositivas muestran que el funcionamiento del analizador LL(1) se basa en una **tabla de análisis sintáctico**.

Cada fila representa un símbolo no terminal, mientras que cada columna corresponde a un símbolo terminal del lenguaje. En cada intersección aparece la producción que debe aplicarse cuando el analizador se encuentra en dicha situación.

Durante el análisis, el compilador consulta esta tabla para decidir automáticamente la siguiente producción, evitando realizar búsquedas innecesarias.

Esta estrategia permite construir analizadores predictivos de manera relativamente sencilla.

Ventajas de LL(1)

Entre las principales ventajas del método LL(1) pueden destacarse:

- simplicidad de implementación;
- análisis rápido;
- construcción relativamente sencilla de la tabla de análisis;
- detección inmediata de numerosos errores sintácticos.

No obstante, estas ventajas también implican ciertas restricciones sobre la forma que puede adoptar la gramática.

7.5 Análisis sintáctico ascendente

Las diapositivas presentan posteriormente un enfoque diferente conocido como **análisis ascendente**.

En este método el proceso comienza con la cadena de entrada y, mediante reducciones sucesivas, intenta reconstruir el símbolo inicial de la gramática.

Mientras que el análisis descendente intenta generar la cadena, el análisis ascendente busca identificar progresivamente las producciones utilizadas para construirla.

Este enfoque resulta considerablemente más poderoso y permite analizar una mayor cantidad de gramáticas.

7.6 Gramáticas LR(0)

Dentro del análisis ascendente, las diapositivas estudian las **gramáticas LR(0)**.

Al igual que LL(1), la denominación describe el funcionamiento del algoritmo:

- lectura de izquierda a derecha;
- construcción de una derivación por la derecha en sentido inverso;
- utilización de cero símbolos adicionales de anticipación.

El analizador LR(0) emplea una colección de estados que representan diferentes situaciones durante el reconocimiento de la cadena.

Cada estado contiene elementos conocidos como **ítems LR**, los cuales indican el progreso alcanzado dentro de cada producción.

Construcción de los estados LR

Las diapositivas muestran el procedimiento mediante el cual se construyen los estados del analizador LR(0).

Inicialmente se genera el estado correspondiente al símbolo inicial de la gramática.

Posteriormente se aplican operaciones de cierre y transición para obtener nuevos estados, formando finalmente un autómata capaz de reconocer todas las cadenas válidas del lenguaje.

Cada transición representa el avance dentro de una producción específica.

Tabla LR(0)

Una vez construido el autómata LR, es posible generar la tabla utilizada por el analizador sintáctico.

Esta tabla contiene dos componentes principales:

- acciones (*Action*);
- transiciones (*Goto*).

Las acciones indican si el analizador debe:

- desplazar (*shift*);
- reducir (*reduce*);
- aceptar (*accept*);
- reportar un error.

Por su parte, la tabla **Goto** determina el siguiente estado cuando se reconoce un símbolo no terminal.

7.7 GNU Bison

Después de estudiar los fundamentos del análisis sintáctico, las diapositivas presentan **GNU Bison**, una herramienta destinada a generar automáticamente analizadores sintácticos.

Al igual que Lex automatiza el análisis léxico, Bison automatiza la construcción del parser.

El desarrollador únicamente debe proporcionar la gramática del lenguaje junto con las acciones asociadas a cada producción.

Bison genera posteriormente el código correspondiente al analizador sintáctico.

Estructura de un archivo Bison

Las diapositivas indican que un archivo de Bison se encuentra dividido en varias secciones principales:

- declaraciones;
- definición de tokens;
- reglas gramaticales;
- código auxiliar.

Las reglas gramaticales constituyen la parte más importante del archivo, ya que describen completamente la sintaxis del lenguaje.

Cada producción puede incluir acciones que serán ejecutadas cuando la regla sea reconocida correctamente.

7.8 Integración entre Lex y Bison

El proceso completo de compilación puede entenderse como la colaboración entre ambas herramientas.

En primer lugar, **Lex** procesa el programa fuente y transforma la secuencia de caracteres en tokens.

Posteriormente, **Bison** recibe dichos tokens y verifica que aparezcan conforme a las reglas definidas por la gramática.

Si ambas etapas concluyen satisfactoriamente, el compilador puede continuar con las fases posteriores del procesamiento del programa.

Esta integración demuestra cómo todos los conceptos estudiados durante la materia se encuentran estrechamente relacionados: las expresiones regulares permiten construir analizadores léxicos; los autómatas hacen posible el reconocimiento de patrones; las gramáticas describen la estructura del lenguaje; y los

analizadores sintácticos verifican que dicha estructura sea respetada.

Referencias

Aguilar J., F. J. (2026). Presentaciones del curso de Teoría de Lenguajes y Programación (Temas: Teoría de Lenguajes, Alfabetos, Gramáticas, BNF, Jerarquía de Chomsky, Expresiones Regulares, AFD, AFND, Algoritmo de Thompson, Método de Subconjuntos, Lex, Árboles de Derivación, Gramáticas LL(1), Gramáticas LR(0) y GNU Bison) [Presentaciones en PDF]. Facultad de Ingeniería, Universidad Panamericana.